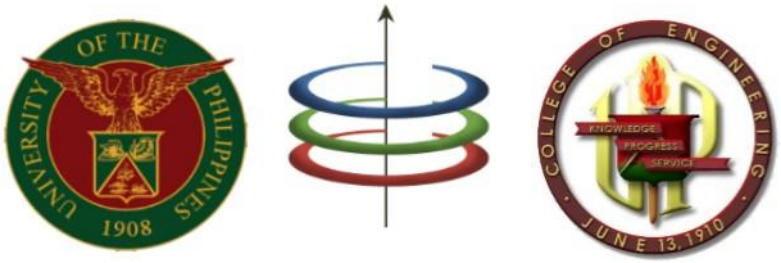
EEE 121 Lecture 10b:
Problem Solving Approaches

Dynamic Programming



Algorithm Designs

1. **Divide-and-Conquer:**
   - Break up a problem into independent subproblems
   - Solve each subproblem then combine mini-solutions to solve original

2. **Greedy**:
   - Build up a solution incrementally
   - Greedy choice – local best action leads to globally-optimal solution

3. **Dynamic Programming (OUR FOCUS):**
   - Break up a problem into a series of overlapping subproblems
   - Build up optimal solution from solutions of subproblems

NOTE: *independent subproblems (D&C) vs. overlapping subproblems (DP)*

Example: Solving n-th Fibonacci

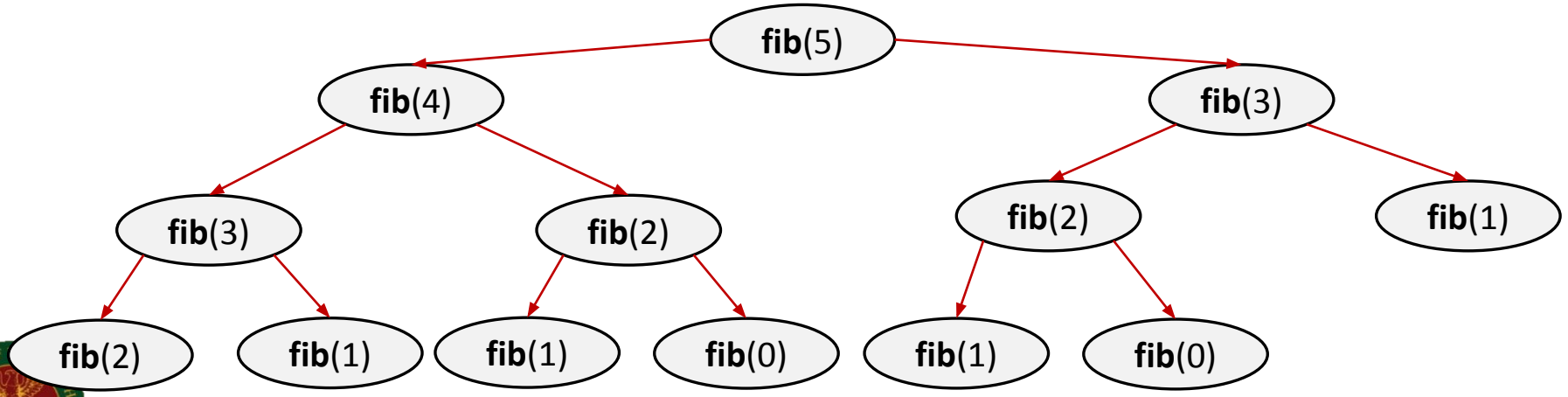
- Consider the following recursive procedure:

```
int fib( int n ) {  //T(n)
    if ( n == 0 ) {
        return 0;          //O(1)
    } else if (n == 1) {
        return 1;          //O(1)
    } else {
        return fib(n - 1) + fib(n - 2);      //T(n - 1) + T(n - 2)+O(1)
    }
}
```

Complexity is approximately $O(2^n)$

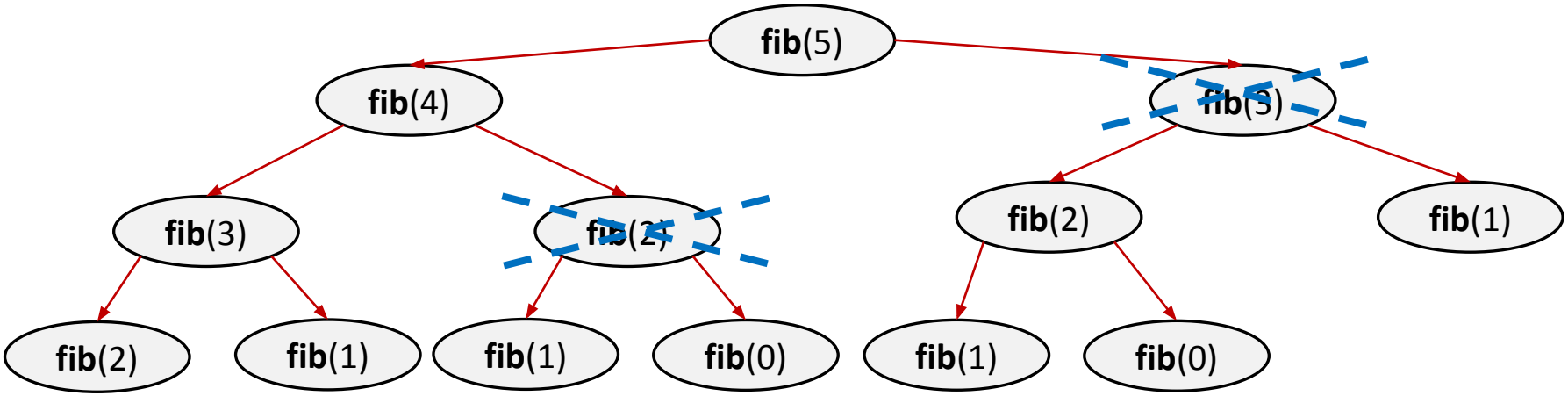
Can we do better?

- Graphical Illustration of recursive calls:



Example: Solving n-th Fibonacci

- **Observation:** If we have solved a subproblem, we don't need to solve it again!



- Cache the answer when a subproblem is solved
  - **Benefit:** reduces # of computation steps (↓ time complexity)
  - **Cons:** Need memory to store mini-results (↑ space complexity)

Dynamic Programming Approaches

1. **Top-down Approach**
   - recurses to solve smaller problems, which recurse to solve even smaller problems
   - Cache the return values of recursive calls (Memoization)

2. **Bottom-up Approach**
   - iterates through problems by size; solving the small problems first
   - Fill-up a table with computations; start from smallest problem size going towards largest problem size (Tabulation)

Top-down vs. Bottom-up for Fib(n)

Top-down Approach:

```
int memo[20];

int fib( int n ) {
    if ( n == 0 ) {
        return 0;
    } else if (n == 1) {
        return 1;
    } else if(memo[n] != -1) {
        return memo[n];
    }

    memo[n] = fib(n - 1)
              + fib(n - 2);
    return memo[n];
}
```

Bottom-up Approach:

```
int fib( int n ) {
    int table[n+1];

    table[0] = 0;
    table[1] = 1;

    for(int i = 2; i <= n; i++) {
     table[i] = table[i - 1]
                    + table[i-2];
    }

    return table[n]
}
```

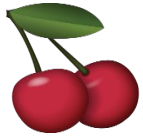
What is the time complexity?

Knapsack (KS) Problem

- **Given:**
  - a knapsack with integer capacity **C**

Capacity = 10

  - n items with integer weights **{w_1, w_2, …, w_n}** and values **{v_1, v_2, …, v_n}**

item:					
weight:	6	2	4	3	11
value:	20	8	14	13	35

- **Goal:** a subset **S** of n items such that the total weight fits in the knapsack and total value is maximized

$$\max \sum_{i \in S} v_i \quad such\ that \sum_{i \in S} w_i \leq C$$

Two Variants of KS Problem

1. **Unbounded Knapsack**
   - Suppose I have infinite copies of all items. What's the most valuable way to fill the knapsack?

2. **0/1 Knapsack**
   - Suppose I only have one copy of each item. What's the most valuable way to fill the knapsack?

- Suppose we try a greedy algorithm to solve this:
  - **Greedy choice:** pick item with largest value that fits in the knapsack

	1		2		
weight:	6	2	4	3	11
value:	20	8	14	13	35

C = 10

Solving Unbounded Knapsack

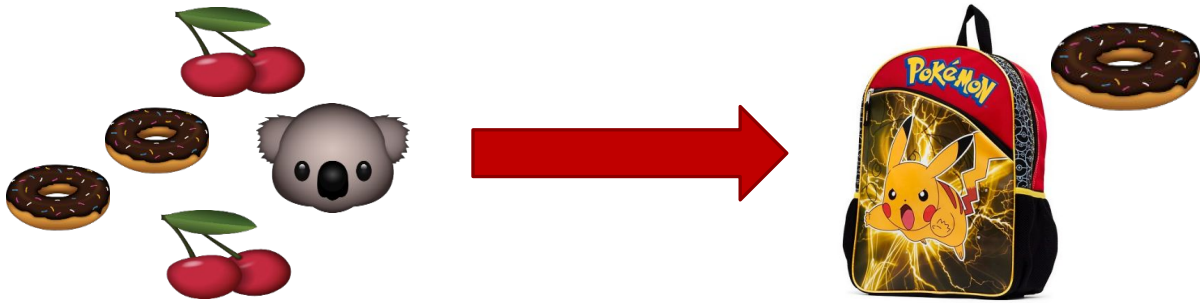
- Steps in dynamic programming

1. Identify optimal substructure with overlapping subproblems.
2. Define a recursive formulation.
3. Use dynamic programming to solve the problem.
   - Either top-down (via memorization) or bottom-up (via tabulation)

Solving Unbounded Knapsack

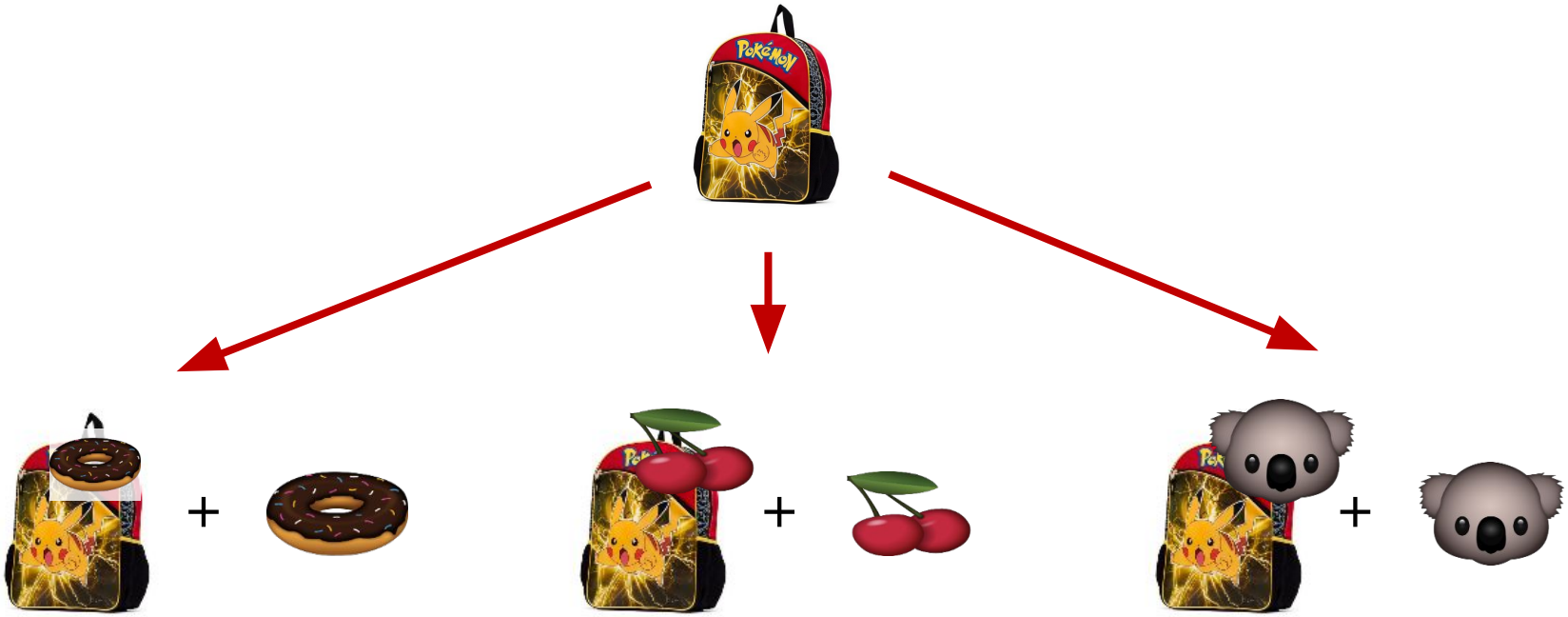
1. Identify optimal substructure with overlapping subproblems
   A. Suppose the subset **S** below is the optimal solution for capacity C

   B. Subset **S**\{j} is the optimal solution for capacity C - w_j



Solving Unbounded Knapsack

1. Identify optimal substructure with overlapping subproblems



Finding the maximum value from all the possible subproblems

Solving Unbounded Knapsack

2. Define a recursive function

Let **KS(x)** be a function that returns the optimal value for capacity x.

$$KS(x) = \max_i \{ \text{  } + value\{\text{  }\} \}$$

The maximum over all i such that $w_i \leq x$.

The optimal way to fill the smaller knapsack with weight $x - w_i$

value of item i

$$KS(x) = \begin{cases} 0, & if\ no\ w_i \leq x \\ \max_i KS(x - w_i) + v_i, & otherwise \end{cases}$$

Solving Unbounded Knapsack

3. Use dynamic programming (top-down)

```
int memo[capacity+1]; //initialized to -1
int v[n];
int w[n];

int KS1(int x) {
    int ans = 0;

    if(memo[x] != -1)
     return memo[x];

    for(int i = 0; i < n; i++) {
        if(w[i] <= x)
            ans = max(ans,KS1(x-w[i]) + v[i]);
    }

    memo[x] = ans;    //cached solution
    return ans;
}
```

$$KS(x) = \begin{cases} 0, & \text{if no } w_i \leq x \\ \max_i KS(x - w_i) + v_i, & \text{otherwise} \end{cases}$$

Solving Unbounded Knapsack

3. Use dynamic programming (bottom-up)

```
int table[capacity + 1]; //initialized to 0
int v[n];
int w[n];

int KS1(int capacity) {

    for(int x = 0; x <= capacity; x++) {
        for(int i = 0; i < n; i++) {
            if(w[i] <= x)
                table[x] = max(table[x],table[x-w[i]] + v[i]);
        }
    }

    return table[capacity];
}
```

$$KS(x) = \begin{cases} 0, & \text{if no } w_i \leq x \\ \max_i KS(x - w_i) + v_i, & \text{otherwise} \end{cases}$$

From Unbounded KS to 0/1 KS

- Since an item should be included at most once, we need to track which items have been considered
- **Step 1:** Optimal substructure w/ overlapping subproblem is the same as unbounded KS
- **Step 2:** Recursive function

$$no\ item \Longrightarrow KS(0,x) = 0$$

If item does not fit, skip

$$KS(n,x) = \begin{cases} 0 & , \quad n = 0 \\ KS(n-1,x) & , \quad x < w_n \\ \max\{KS(n-1,x), KS(n-1,x-w_n) + v_n\} & , \quad otherwise \end{cases}$$

Choice 1: do not include n-th item in KS

Choice 2: include n-th item in KS

From Unbounded KS to 0/1 KS

- **Step 3:** Use dynamic programming (top-down)

```
int memo[n+1][capacity+1]; //initialized to -1
int v[n+1];   //{v1, v2, …, vn}
int w[n+1];         //{w1, w2, …, wn}

int KS2(int n, int x) {
    int ans = 0;
    if(n = 0)
        return 0;
    else if(w[n] > x)
        ans = KS2(n-1,x);
    else if(memo[n][x] != -1)
        return memo[n][x];
    else
        memo[n][x] = max(KS2(n-1,x),KS2(n-1,x – w[n]) + v[n]);

    memo[n][x] = ans;   //cached solution
    return ans;
}
```

From Unbounded KS to 0/1 KS

- Bottom-up version of the code is left as an exercise
- Given:

	1	2	3	4	5
item:					
weight:	6	2	4	3	11
value:	20	8	14	13	35

Capacity = 10

Contents of table[][] for bottom-up

n	x = 0	x = 1	x = 2	x = 3	x = 4	x = 5	x = 6	x = 7	x = 8	x = 9	x = 10
0	0	0	0	0	0	0	0	0	0	0	0
1											
2											
3											
4											
5											

From Unbounded KS to 0/1 KS

- Bottom-up version of the code is left as an exercise
- Given:

	1	2	3	4	5
item:					
weight:	6	2	4	3	11
value:	20	8	14	13	35

Capacity = 10

Contents of table[][] for bottom-up

n	x = 0	x = 1	x = 2	x = 3	x = 4	x = 5	x = 6	x = 7	x = 8	x = 9	x = 10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	20	20	20	20	20
2											
3											
4											
5											

From Unbounded KS to 0/1 KS

- Bottom-up version of the code is left as an exercise
- Given:

	1	2	3	4	5
item:					
weight:	6	2	4	3	11
value:	20	8	14	13	35

Capacity = 10

Contents of table[][] for bottom-up

n	x = 0	x = 1	x = 2	x = 3	x = 4	x = 5	x = 6	x = 7	x = 8	x = 9	x = 10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	20	20	20	20	20
2	0	0	8	8	8	8	20	20	28	28	28
3											
4											
5											

From Unbounded KS to 0/1 KS

- Bottom-up version of the code is left as an exercise
- Given:

	1	2	3	4	5
item:					
weight:	6	2	4	3	11
value:	20	8	14	13	35

Capacity = 10

Contents of table[][] for bottom-up

n	x = 0	x = 1	x = 2	x = 3	x = 4	x = 5	x = 6	x = 7	x = 8	x = 9	x = 10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	20	20	20	20	20
2	0	0	8	8	8	8	20	20	28	28	28
3	0	0	8	8	14	14	22	22	28	28	34
4											
5											

From Unbounded KS to 0/1 KS

- Bottom-up version of the code is left as an exercise
- Given:

	1	2	3	4	5
item:					
weight:	6	2	4	3	11
value:	20	8	14	13	35

Capacity = 10

Contents of table[][] for bottom-up

n	x = 0	x = 1	x = 2	x = 3	x = 4	x = 5	x = 6	x = 7	x = 8	x = 9	x = 10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	20	20	20	20	20
2	0	0	8	8	8	8	20	20	28	28	28
3	0	0	8	8	14	14	22	22	28	28	34
4	0	0	8	13	14	21	22	27	28	35	35
5											

From Unbounded KS to 0/1 KS

- Bottom-up version of the code is left as an exercise
- Given:

	1	2	3	4	5
item:					
weight:	6	2	4	3	11
value:	20	8	14	13	35

Capacity = 10

Contents of table[][] for bottom-up

n	x = 0	x = 1	x = 2	x = 3	x = 4	x = 5	x = 6	x = 7	x = 8	x = 9	x = 10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	20	20	20	20	20
2	0	0	8	8	8	8	20	20	28	28	28
3	0	0	8	8	14	14	22	22	28	28	34
4	0	0	8	13	14	21	22	27	28	35	35
5	0	0	8	13	14	21	22	27	28	35	35

Tracing the DP Solution in KS

The memo[][] or table[][] can help us trace the path to reach the DP solution in 0/1 KS

- memo[n][x] == memo[n – 1][x] ⟹ do not include item n in KS
- Otherwise, include item n in KS and let x = x – w[n]

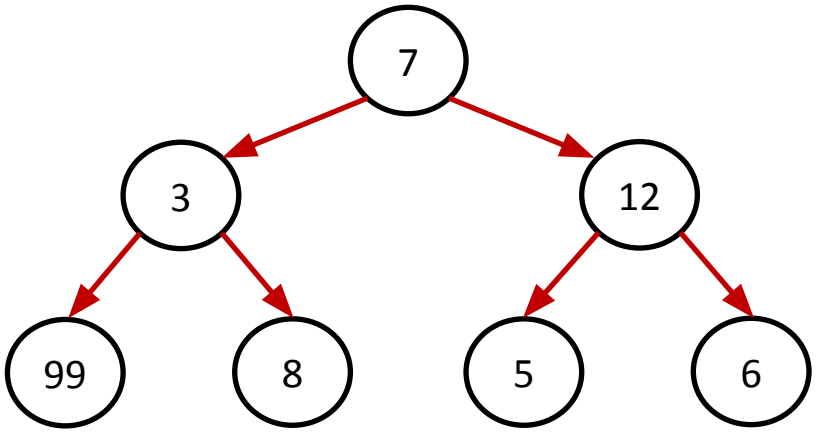
item:	**1**	**2**	**3**	**4**	**5**
weight:	6	2	4	3	11
value:	20	8	14	13	35

n	x = 0	x = 1	x = 2	x = 3	x = 4	x = 5	x = 6	x = 7	x = 8	x = 9	x = 10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	20	20	20	20	20
2	0	0	8	**8**	8	8	20	20	28	28	28
3	0	0	8	8	14	14	22	**22**	28	28	34
4	0	0	8	13	14	21	22	27	28	35	**35**
5	0	0	8	13	14	21	22	27	28	35	35

Maximum Path Cost in a Tree

- Recall the last problem in the greedy lecture

  Problem: Find the path from root to any leaf node such that sum of node values along the path is maximized



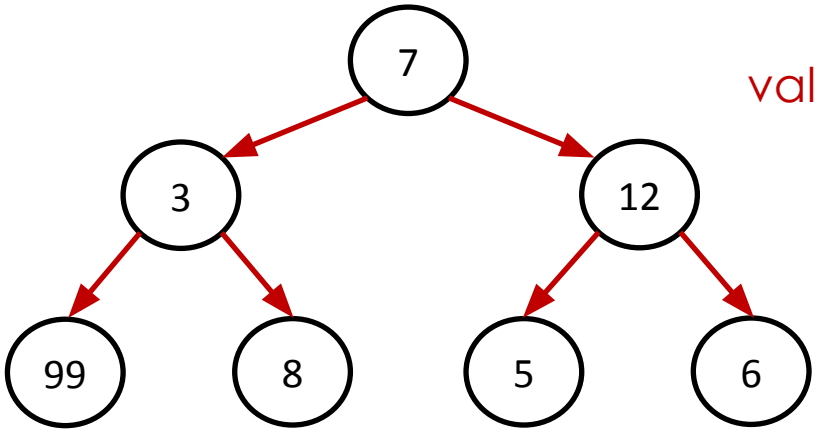
Greedy solution: 7 □ 12 □ 6

Optimal solution: 7 □ 3 □ 99

- Can we apply dynamic programming on a tree structure?

Maximum Path Cost in a Tree

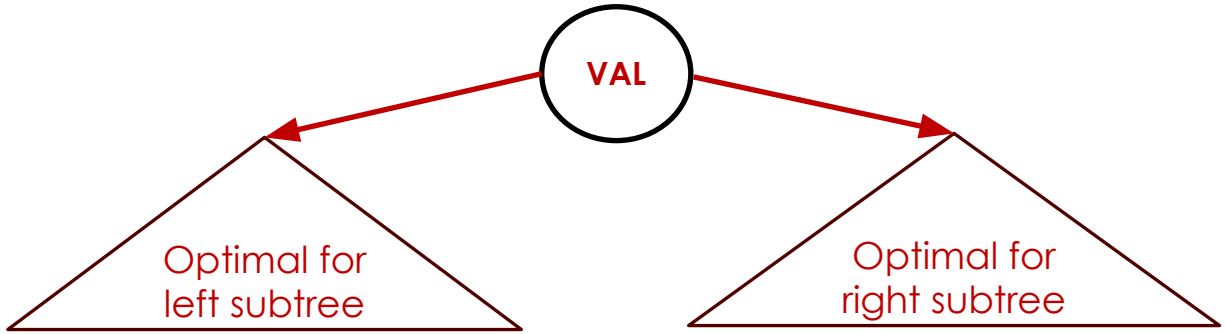
- Array implementation of tree DS:



val[i]	7	3	12	99	8	5	6
	0	1	2	3	4	5	6

- Left child of node i @ index 2i+1
- Right child of node i @ index 2i+2

- **Step 1:** optimal substructure with overlapping subproblems



Maximum Path Cost in a Tree

- **Step 2:** Define recursive function

$$COST(i) = \begin{cases} val[i] & i \geq 2^{height} \\ val[i] + \max\{COST(2i+1), COST(2i+2)\}, & otherwise \end{cases}$$

Leaf nodes

Value at node i

Cost of left subtree

Cost of right subtree

- **Step 3:** Use dynamic programming (do this on your own ☺)

Maximum Path Cost in a Tree

- **Step 3:** Use dynamic programming (top-down)

```
int val[];  // Stores values at nodes
int memo[];   // Memoization array

int maxPathCost(int i, int height, int n) {
    // Base Case: if i is greater than the height or exceeds array size
    // (1<<height is equal to 2^height)
    if (i >= n || i >= (1 << height)) {
        return (i < n) ? val[i] : 0;
    }

    if (memo[i] != -1) return memo[i]; // Caching

    int left = maxPathCost(2 * i + 1, height, n);
    int right = maxPathCost(2 * i + 2, height, n);

    memo[i] = val[i] + max(left, right);
    return memo[i];
}
```

DP for Counting Problems

- **Setup:** You are running up a staircase with N steps, and can only hop(jump) by odd-numbered steps at a time (either 1 step, 3 steps, ..., 2K+1 for some given integer K).
- **Goal:** Count how many possible ways you can run up to the stairs.
- Recurrence Relation:

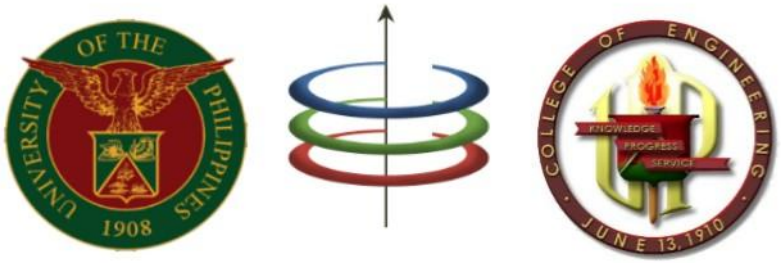
$$f(n) = \begin{cases} . \quad 1 \qquad\qquad , & n = 0 \\ \sum_{i=0}^{\lfloor \frac{n-1}{2} \rfloor} f\big(n - (2i+1)\big), & otherwise \end{cases}$$

- Memo[] for n = 0 to n = 6

1	1	1	2	3	5	8
0	1	2	3	4	5	6

EEE 121 Lecture 10b: **Problem Solving Approaches**

Dynamic Programming



Reminders before we part ways

- **Final Exam on May 26, 6PM-8PM**
  - Graph Representation and Algorithms
    - Traversals
    - Shortest Path Problem
    - Minimum Spanning Trees
  - Problem Solving Approaches
    - Greedy Algorithms
    - Dynamic Programming
- **SET Deadline on May 22 (TODAY)**
  - Answer the Qualtrics Form: https://upsystemdiliman.qualtrics.com/jfe/form/SV_0j0jnerWjOr2Ele
  - Instructor: Steven Sison

I know that our subject is hard, but I hope that you learned a lot during our lectures and class activities :D

See you again soon! Hopefully not in EEE 121 again.